



UniswapV3 Independent Audit Report

Version 1.0

Audited by:

LAU LOK JING

August 6, 2026

Table of Contents

- Table of Contents
- Protocol Summary
 - 1. Protocol Features
 - 2. Potential Security Issues in Integration & Interaction
 - * 2.1 Reliance on `slot0` for Spot Price or Low `observationCardinality` V3 Pool Results in Price Oracle Manipulation
 - Attack Vector: Read-only Reentrancy
 - * 2.2 Liquidity Fragmentation results in Fee Tier Manipulation
 - Attack Vector: Uninitialized Pool Price Oracle Manipulation
 - * 2.3 Liquidity Provider Risks
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- Low
 - [L-1]: Gas Griefing in `UniswapV3Pool::swap` via Tick Spamming on Low-Gas Rollups may result in Denial of Service (DoS)
 - [L-2]: Missing Input Validation in `SwapRouter` on `amountOutMinimum` in `exactInputSingle` and `exactInput` and `amountInMaximum` in `exactOutputSingle` and `exactOutput`, will result in MEV or sandwich attacks on user error
 - [L-3]: `SwapRouter::exactInputInternal` and `SwapRouter::exactOutputInternal` use `address(0)` as Sentinel for `address(this)`, potentially result in user fund loss if `PeripheryPayments::sweepToken` is not called at the end of the swap.
 - [L-4]: Lack of Input Validation In `NonfungiblePositionManager`, `mint`, `increaseLiquidity`, and `decreaseLiquidity`, Allows `amount0Min = 0` and `amount1Min = 0`, thus enabling MEV/sandwich griefing attacks on user error
- Informational

- [I-1]: Non-Atomic Deployment and Initialization of Pools Creates Front-Running Risk
- [I-2]: `PoolAddress::computeAddress` uses precomputed `POOL_INIT_CODE_HASH` will not work on chains that has non-standard `CREATE2` implementations, leading to invalid address computed

Protocol Summary

1. Protocol Features

Core/Periphery Architecture:

V3 is a smart contract that consists of two layers:

- **Core contracts** (`v3-core`): Define pool deployment logic, pools states, and asset interactions. Core contracts provide fundamental safety guarantees.
- **Periphery contracts** (`v3-periphery`): A periphery layer written on top of `v3-core` to provide safety and to increase clarity to users. Key periphery contracts include `SwapRouter` for single and multi-hop swaps and `NonfungiblePositionManager` for duplication liquidity position creation under same ranges for a single address, adjustment, and exiting. However, NFT is not technically required to provide liquidity; it is purely a product-level abstraction for the Ethereum ecosystem.

Concentrated Liquidity:

Uniswap V3's core innovation is **concentrated liquidity**, which allows liquidity providers (LPs) to allocate capital within specific price ranges rather than across the entire price curve as in V2. This dramatically improves capital efficiency—LPs can focus liquidity around the current market price, enabling traders to execute larger trades with less price impact.

Tick System:

The price range is divided into discrete `ticks`. A change of 1 `tick` represents a relation of $t = \log_{1.0001} P$ to price. P . Each pool has a `tickSpacing` that determines minimum interval between each `tick`, where the higher the `tickSpacing` the higher the pool `fee`. Liquidity providers (LPs) are allowed to define their positions price range by choosing lower and upper ticks.

Multiple Fee Tiers:

Unlike V2's single 0.30% `fee`, V3 offers multiple fee tiers where the defaults configuration are 0.05%, 0.30%, 1%, thereby allowing LPs to choose appropriate fees based on the volatility of the trading pair.

Range Orders:

V3 introduces range orders due to `Tick` system, enabling LPs to place orders that execute only when the price enters a specified range—functioning like limit orders.

Per Pool Oracle:

All Uniswap V3 pools are serving as oracles if queried, providing historical prices and liquidity data. Historical data is stored as observations, and the oracle/data length can be expanded up to 65,535

checkpoints equivalent to at least 9 days of data at the cost of gas. This feature create a time-weighted arithmetic mean price (TWAP) that is highly resistance to price manipulation.

Decentralised & Non-Upgradable & Permissionless:

The protocol is designed as non-upgradable smart contracts that prioritise decentralization and censorship resistance while secure, and without trusted intermediaries.

2. Potential Security Issues in Integration & Interaction

In general, the core contracts are designed with low-level functions that allow developers to maximise integration flexibility, which also means it demands sufficiently deep understanding on `v3-core`.

The periphery contracts exhibit a consistent lack of positive-value validation on critical slippage parameters. Specifically, `SwapRouter` allows `amountOutMinimum` = 0, and `NonfungiblePositionManager` allows `amount0Min` = 0 and `amount1Min` = 0. Both instances expose users to MEV sandwich attacks through `sweepToken` function. While the root cause is similar, the affected contracts and remediation steps differ, so they are documented as separate findings below.

2.1 Reliance on `slot0` for Spot Price or Low `observationCardinality` V3 Pool Results in Price Oracle Manipulation

`slot0` is the first storage slot of a Uniswap V3 pool, containing `sqrtPriceX96`—the current price. It is the latest data point and can be manipulated easily via MEV bots and flash loans. Protocols that use `slot0` instead of TWAP for price calculations expose users to significant price manipulation vulnerabilities. If `observationCardinality` is set to zero, TWAP protection is bypassed entirely, making the protocol susceptible to single-block price manipulation.

Attack Vector: Read-only Reentrancy

1. Call a flashloan of \$500,000 Stablecoins.
2. Call `swap` on UniswapV3Pool to inflate `TokenX` price by 1000% (easier due to concentrated liquidity), receiving \$250,000 `tokenX` due to priceImpact,
3. During `uniswapV3SwapCallback`, call an external lending protocol to borrow `USDC` using `tokenX`,
4. Protocol reads uniswapV3 pool with low `cardinality`, valuing it at 500-1000% increment, assume overcollateralize at 200%, \$250,000 is valued at \$1,250,000
5. An amount of \$625,000 is taken out from the lending protocol, pay back \$500,000 of flashloan, total profit = \$625,000 - (\$500,000 + flashloanfees)

2.2 Liquidity Fragmentation results in Fee Tier Manipulation

Each fee tier is a separate pool with its own address, its own liquidity, and its own price. This means for [WETH/USDC](#), there can be up to four distinct pools, each with a different fee.

Attack Vector: Uninitialized Pool Price Oracle Manipulation The Setup:

An external protocol (e.g., a lending or yield aggregator) uses a Uniswap V3 TWAP oracle to price assets. The protocol hardcodes a specific fee tier (e.g., 0.05%) for a given pair.

The Attack:

1. The attacker checks if the pool for that fee tier has not yet been deployed.
2. The attacker calls `UniswapV3Factory.createPool` to deploy the pool.
3. The attacker calls `pool.initialize` with a manipulated initial price.
4. The external protocol queries the oracle, which reads from this newly deployed, manipulated pool.
5. The protocol misprices assets, allowing the attacker to borrow, liquidate, or trade at a profit.

2.3 Liquidity Provider Risks

1. Inactive Positions: If the price moves outside the LP's chosen range, the position becomes inactive and stops earning fees until the price returns.
2. NFT Position Representation: V3 liquidity positions are represented as ERC-721 NFTs. Staking requires transferring the NFT to a contract, which means transferring the right of the liquidity to another contract.
3. Just-In-Time (JIT) liquidity providers can inject liquidity into a pool just before a trade and remove it immediately after, siphoning up to a significant portion of passive LP fee revenue per trade through flashloan. This harms long-term LPs who provide stable liquidity.

Disclaimer

I Lau Lok Jing makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by me is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

I use the CodeHawks severity matrix to determine severity. See the documentation for more details. I reserve severity Critical - CRIT for direct theft of fund.

Audit Details

The findings described in this document correspond to the following repositories:

1. Uniswap/v3-core (hash: d0831dc6b8a318df3872b6d68f6de135c9f3ec29): github.com/Uniswap/v3-core
2. Uniswap/v3-periphery (hash: 0682387198a24c7cd63566a2c58398533860a5d1): <https://github.com/Uniswap/v3-periphery>

Scope

```
1  Uniswap/v3-core:
2  |-- interfaces
3  |   |-- callback
4  |       |-- IUniswapV3FlashCallback.sol
5  |       |-- IUniswapV3MintCallback.sol
6  |       |-- IUniswapV3SwapCallback.sol
```

```

7      |-- pool
8      |-- IUniswapV3PoolActions.sol
9      |-- IUniswapV3PoolDerivedState.sol
10     |-- IUniswapV3PoolEvents.sol
11     |-- IUniswapV3PoolImmutableables.sol
12     |-- IUniswapV3PoolOwnerActions.sol
13     |-- IUniswapV3PoolState.sol
14     |-- IERC20Minimal.sol
15     |-- IUniswapV3Factory.sol
16     |-- IUniswapV3Pool.sol
17     |-- IUniswapV3PoolDeployer.sol
18 |-- libraries
19 |-- BitMath.sol
20 |-- FixedPoint128.sol
21 |-- FixedPoint96.sol
22 |-- FullMath.sol
23 |-- LiquidityMath.sol
24 |-- LowGasSafeMath.sol
25 |-- Oracle.sol
26 |-- Position.sol
27 |-- SafeCast.sol
28 |-- SqrtPriceMath.sol
29 |-- SwapMath.sol
30 |-- Tick.sol
31 |-- TickBitmap.sol
32 |-- TickMath.sol
33 |-- TransferHelper.sol
34 |-- UnsafeMath.sol
35 |-- NoDelegateCall.sol
36 |-- UniswapV3Factory.sol
37 |-- UniswapV3Pool.sol
38 |-- UniswapV3PoolDeployer.sol
39
40 Uniswap/v3-periphery:
41 |-- base
42 |-- BlockTimestamp.sol
43 |-- ERC721Permit.sol
44 |-- LiquidityManagement.sol
45 |-- Multicall.sol
46 |-- PeripheryImmutableState.sol
47 |-- PeripheryPayments.sol
48 |-- PeripheryPaymentsWithFee.sol
49 |-- PeripheryValidation.sol
50 |-- PoolInitializer.sol
51 |-- SelfPermit.sol
52 |-- interfaces
53 |-- external
54 |-- IERC1271.sol
55 |-- IERC20PermitAllowed.sol
56 |-- IWETH9.sol
57 |-- IERC20Metadata.sol

```



```

58      |-- IERC721Permit.sol
59      |-- IMulticall.sol
60      |-- INonfungiblePositionManager.sol
61      |-- INonfungibleTokenPositionDescriptor.sol
62      |-- IPeripheryImmutableState.sol
63      |-- IPeripheryPayments.sol
64      |-- IPeripheryPaymentsWithFee.sol
65      |-- IPoolInitializer.sol
66      |-- IQuoter.sol
67      |-- IQuoterV2.sol
68      |-- ISelfPermit.sol
69      |-- ISwapRouter.sol
70      |-- ITickLens.sol
71      |-- IV3Migrator.sol
72      |-- libraries
73      |-- BytesLib.sol
74      |-- CallbackValidation.sol
75      |-- ChainId.sol
76      |-- HexStrings.sol
77      |-- LiquidityAmounts.sol
78      |-- NFTDescriptor.sol
79      |-- NFTSVG.sol
80      |-- OracleLibrary.sol
81      |-- Path.sol
82      |-- PoolAddress.sol
83      |-- PoolTicksCounter.sol
84      |-- PositionKey.sol
85      |-- PositionValue.sol
86      |-- SqrtPriceMathPartial.sol
87      |-- TokenRatioSortOrder.sol
88      |-- TransferHelper.sol
89      |-- NonfungiblePositionManager.sol
90      |-- NonfungibleTokenPositionDescriptor.sol
91      |-- SwapRouter.sol
92      |-- V3Migrator.sol

```

- Solc Version: =0.7.6
- Chain(s) to deploy contract to: Ethereum, ZKSync, Polygon, Optimish

Roles

- Owner: The owner of the Uniswap V3 Pool Factory, authorised to enable new fee tiers.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest, which position could be represent by a NFT.
- User: A user who swap or takes out flash loans from the protocol.

Executive Summary

A total of **6** risks are found which composed of **4** low, and **2** informational. About **105** hours are spent on the findings using Static Analysis tools from Slither, manual review, and fuzzing through Foundry.

The report is carried out in three phases: 1. In the first day, I focused on gaining a high-level understanding of the protocol by reviewing its whitepaper to understand the technical implementations. 2. In the following days, manual review of the state invariants, the flash loan function, and pool initialization process are performed.

Issues found

Severity	Number of issues found
High	0
Medium	0
Low	4
Info	2
Gas Optimizations	0
Total	6

Findings

Low

[L-1]: Gas Griefing in `UniswapV3Pool::swap` via Tick Spamming on Low-Gas Rollups may result in Denial of Service (DoS)

Descriptions:

`UniswapV3Pool::swap` executes a state-modifying while loop to scan, cross, and update active initialized ticks sequentially to calculate price slippage. On modern Layer 2 rollups (such as the OP Stack), individual transaction execution limits are tightly bound by a maximum per-transaction gas cap (typically $2 * 10^{24}$ or 16,777,216 gas) to protect the sequencer from compute exploitation.

`UniswapV3Factory::enableFeeAmount` allows `owner` to configure `tickSpacing` to as low as 1 which matches the whitepaper where different fee and tickSpacing tiers could be added through governance.

```
1  UniswapV3Pool.sol:
2      function swap(
3          address recipient,
4          bool zeroForOne,
5          int256 amountSpecified,
6          uint160 sqrtPriceLimitX96,
7          bytes calldata data
8      ) external override noDelegateCall returns (int256 amount0, int256
          amount1) {
9          ...
10         // continue swapping as long as we haven't used the entire
            input/output and haven't reached the price limit
11  @>    while (state.amountSpecifiedRemaining != 0 && state.
            sqrtPriceX96 != sqrtPriceLimitX96) {
12         StepComputations memory step;
13         ...
14     }
15 }
16
17 UniswapV3Factory.sol:
18     function enableFeeAmount(uint24 fee, int24 tickSpacing) public
        override {
19         require(msg.sender == owner);
20         require(fee < 10000000);
21         // tick spacing is capped at 16384 to prevent the situation
            where tickSpacing is so large that
22         // TickBitmap#nextInitializedTickWithinOneWord overflows int24
            container from a valid tick
```

```
23      // 16384 ticks represents a >5x price change with ticks of 1
        bips
24  @>    require(tickSpacing > 0 && tickSpacing < 16384);
25        require(feeAmountTickSpacing[fee] == 0);
26
27        feeAmountTickSpacing[fee] = tickSpacing;
28        emit FeeAmountEnabled(fee, tickSpacing);
29    }
```

Consequently, an attacker can exploit this architectural mismatch by batch-minting 1-wei dust liquidity positions across thousands of consecutive ticks directly above or below the current trading price. When an organic user attempts a large swap that drives the price into the spammed zone, the transaction is forced to process thousands of iterations of the tick-crossing logic. Because each initialized tick cross incurs a fixed computation cost (~13,500 gas for storage updates and bitmap manipulation), the transaction runs out of gas and reverts before completing the swap.

Impact:

1. Directional Price Wall: The pool becomes incapable of adjusting its price past a certain boundary, effectively freezing the rate in one direction.
2. Capital Disruption: LPs in the pool cannot have their liquidity utilized, and traders face persistent transaction failures (Out of Gas errors).
3. Routing Blacklist: Decentralized exchange aggregators (like 1inch or Uniswap Auto Router) will detect the extreme gas overhead during simulation and permanently route user traffic away from the pool, rendering its liquidity pool economically dead.

Proof of Concept:

1. Attacker Pre-computation (Theoretical)
 - Max Per-Transaction Gas Limit on OP Stack = 16,777,216 gas.
 - Average Gas used per single tick crossed $\approx 13,500$ gas.
 - Max Ticks Processed in 1 TX = $\frac{16,777,216}{13,500} \approx 1,242$ ticks.
 - For a tick spacing of 1, an interval of 1,242 ticks allows a price shift of only $\approx 1.25\%$. If the attacker spams 1,500 consecutive ticks, the pool becomes impassable.

Mitigation:

1. Increase Tick Spacing Floor: Modify custom factory parameters on low-gas L2 networks to deprecate tick spacing 1. Enforcing a minimum tick spacing of 10 or 50 significantly multiplies the required price movement distance an attacker must cover, rendering the attack cost-prohibitive.
2. Minimum Liquidity Threshold: Implement a minimum protocol-enforced rule within the `mint` function that scales the required deposit size based on the thickness of the target pool. Rejecting sub-dust (1 wei) mint entries prevents cheap entry spamming. Scaling 1 wei to 100 wei would already raised the griefing cost by 100x.

[L-2]: Missing Input Validation in SwapRouter on `amountOutMinimum` in `exactInputSingle` and `exactInput` and `amountInMaximum` in `exactOutputSingle` and `exactOutput`, will result in MEV or sandwich attacks on user error**Descriptions:**

The `exactInputSingle` and `exactOutputSingle` functions do not enforce a positive minimum output or a finite maximum input. While the contract executes correctly, this creates a periphery/security gap where a user could submit a swap with effectively no slippage protection, making the transaction highly vulnerable to MEV/sandwich attacks. The `checkDeadline` modifier already demonstrates that the router enforces certain safety rails; this finding is an extension of that principle.

While there are justified cases where withdrawal functions are designed to treat `amount = type(uint256).max` as users' intend to fully withdrawn their position, there are no implicit or explicit logical objective to allow a swap function ending state are validated againsts `amountOutMinimum` or `amountInMaximum` equal to zero and `type(uint256).max` respectively.

Steps:

1. A malicious MEV bot listening to tx in the mempool.
2. User mistakenly set `amountOutMinimum = 0` when calling `exactInputSingle` or `exactInput`.
3. The MEV bot sandwiched the user by submitting a tx before the user to inflate the price of the swap tokens.
4. Then, the user swap went through, and user received `amountOut` ≈ 0 , the MEV bot's second tx in the bundle follow and sell the bought token at the highest price, profiting from the price difference.
5. user is left with `amountOut` ≈ 0 .

Impact: Loss of funds via MEV extraction if a user mistakenly sets `amountOutMinimum = 0` in `exactInput/exactInputSingle` or `amountInMaximum = type(uint256).max` in

`exactOutputSingle/exactOutput`.

Mitigation:

Consider implement input validation checks, although this will increase the gas costs per swap, it is significantly less than the one-time user error where a significantly fund could be drained.

Add the following lines into `SwapRouter.sol`:

```
1     function exactInputSingle(ExactInputSingleParams calldata params)
2         ...
3     {
4 +     require(params.amountOutMinimum > 0, 'Invalid min amountOut')
5         amountOut = exactInputInternal(
6             params.amountIn,
7             params.recipient,
8             params.sqrtPriceLimitX96,
9             SwapCallbackData({path: abi.encodePacked(params.tokenIn,
10                 params.fee, params.tokenOut), payer: msg.sender})
11         );
12         require(amountOut >= params.amountOutMinimum, "Too little
13             received");
14     }
15
16     function exactInput(ExactInputParams memory params)
17         ...
18     {
19         address payer = msg.sender; // msg.sender pays for the first
20         hop
21         require(params.amountOutMinimum > 0, 'Invalid min amountOut')
22         ...
23     }
24
25     function exactOutputSingle(ExactOutputSingleParams calldata params)
26         ...
27     {
28 +     require(params.amountInMaximum < type(uint256).max, 'Invalid max
29         amountIn')
30         ...
31     }
32
33     function exactOutput(ExactOutputParams calldata params)
34         ...
35     {
36 +     require(params.amountInMaximum < type(uint256).max, 'Invalid max
37         amountIn')
38         ...
39     }
```

[L-3]: `SwapRouter::exactInputInternal` and `SwapRouter::exactOutputInternal` use `address(0)` as Sentinel for `address(this)`, potentially result in user fund loss if `PeripheryPayments::sweepToken` is not called at the end of the swap.

Descriptions:

The `SwapRouter` uses `address(0)` as a sentinel for `address(this)` to save ~240 gas per swap/hop. This is a micro-optimization that creates a macro-risk. By silently redirecting invalid input, the contract implicitly takes custody of user assets without maintaining any stateful accounting of ownership. Consequently, any asset left unclaimed in the router becomes a public bounty, claimable by the first caller of `PeripheryPayments::sweepToken` or `PeripheryPayments::refundETH`. This design goes against the fundamental security principle of validating external inputs and trusting users to structure transactions perfectly. The negligible gas savings do not justify the significant operational risk for integrators and end-users.

I acknowledge following the design benefits:

1. It reduces calldata costs (zero bytes cost 4 gas vs. 16 gas for non-zero bytes).
2. It enables atomic `Multicall` workflows where intermediate outputs must be held by the router for subsequent operations (i.e., swap -> sweep -> unwrap).

However, this design introduces a significant footgun for integrators and end-users:

1. If a protocol or user passes `address(0)` without intending the router to custody the output, the tokens will remain in the router's balance.
2. Because `sweepToken` and `refundETH` are permissionless functions, any unclaimed assets left in the router beyond the current transaction are publicly accessible and can be claimed by the first caller (including MEV bots).

`SwapRouter.sol`:

```
1     function exactInputInternal(  
2         uint256 amountIn,  
3         address recipient,  
4         uint160 sqrtPriceLimitX96,  
5         SwapCallbackData memory data  
6     ) private returns (uint256 amountOut) {  
7         // allow swapping to the router address with address 0  
8 @>         if (recipient == address(0)) recipient = address(this);  
9  
10        ...  
11  
12        function exactOutputInternal(  
13            uint256 amountOut,  
14            address recipient,
```

```
15         uint160 sqrtPriceLimitX96,  
16         SwapCallbackData memory data  
17     ) private returns (uint256 amountIn) {  
18         // allow swapping to the router address with address 0  
19     @>         if (recipient == address(0)) recipient = address(this);  
20  
21     }
```

Proof of Concept:

1. `userA` call `exactInputSingle` with `recipient = address(0)`,
2. A MEV bot observes the pending swap with `recipient = address(0)`. The bot bundles a second transaction immediately after the swap, calling `sweepToken` to transfer the router's entire token balance to itself,
3. `userA` call `sweepToken`, but because `userA` did not atomically sweep the output in the same transaction, the funds are claimed by the bot,
4. `userA` ends up with `amountOut = 0`

Impact: Loss of user funds via MEV extraction if a user mistakenly sets `recipient = address(0)`

.

Mitigation:

Since UniswapV3 has been deployed and live on Ethereum mainnet and many other chains:

- No changes are recommended to the deployed `SwapRouter` contract, as this behavior is intentional, documented, and relied upon by existing integrations. Modifying it would increase gas costs and break atomic multicall workflows that many other Defi protocols that adhere to it.
- External protocols integrating with `SwapRouter` must explicitly set `recipient` to the intended final address, or be fully aware of the sentinel behavior and use `Multicall` to sweep outputs atomically within the same transaction.
- Frontends and SDKs should clearly warn users if `recipient` is set to `address(0)` or if assets are being sent directly to the router address without an immediate sweep.

[L-4]: Lack of Input Validation In NonfungiblePositionManager, mint, increaseLiquidity, and decreaseLiquidity, Allows amount0Min = 0 and amount1Min = 0, thus enabling MEV/sandwich griefing attacks on user error

Descriptions:

Similar to the `SwapRouter` issue, the `mint` and `increaseLiquidity` functions do not enforce positive values for `amount0Min` and `amount1Min`. A value of 0 effectively signals **100% slippage tolerance**. This bypasses the protocol's built-in slippage protection, allowing an attacker (or MEV bot) to arbitrarily manipulate the pool price immediately before the transaction executes. Furthermore, `V3Migrator::migrate` inherits this risk, as it simply passes these parameters down to `mint`.

```
1 V3Migrator.sol:
2     function migrate(MigrateParams calldata params) external override {
3         ...
4         // mint v3 position
5         (, , uint256 amount0V3, uint256 amount1V3) =
6         @>     INonfungiblePositionManager(nonfungiblePositionManager).
7             mint(
8                 INonfungiblePositionManager.MintParams({
9                     token0: params.token0,
10                    token1: params.token1,
11                    fee: params.fee,
12                    tickLower: params.tickLower,
13                    tickUpper: params.tickUpper,
14                    amount0Desired: amount0V2ToMigrate,
15                    amount1Desired: amount1V2ToMigrate,
16                    amount0Min: params.amount0Min,
17                    amount1Min: params.amount1Min,
18                    recipient: params.recipient,
19                    deadline: params.deadline
20                })
21            );
22            ...
23        }
```

This finding is **structurally identical** to the `SwapRouter` zero-slippage issue (`amountOutMinimum = 0`). Both are documented in this report as separate findings due to their distinct contracts and parameter names, but they share the same root cause and remediation principle.

Impact: Loss of user funds via MEV extraction if a user or an integrator submits a transaction with `amount0Min = 0` and/or `amount1Min = 0` will have their position minted (or increased) at a manipulated price, causing a **direct and measurable loss of funds** equivalent to the price spread. The attacker captures this spread via arbitrage.

The lack of slippage protection enables value extraction in multiple contexts, all stemming from the same root cause:

Scenario	Attack Vector	Victim Context
New Pool Initialization	Attacker front-runs the user's <code>mint</code> on a freshly initialized (shallow) pool, sets a manipulative initial price, executes the mint, and back-runs to restore price.	User migrating from V2 via <code>V3Migrator</code> or minting into a brand-new pool.
Active Pool Manipulation (Griefing)	Attacker performs a standard sandwich attack (front-run swap -> victim mint -> back-run swap), extracting value from the spread.	Any user calling <code>mint</code> or <code>increaseLiquidity</code> on an active pool.
JIT Liquidity Extraction	(Supplementary) An attacker utilizes JIT liquidity to capture fee revenue immediately before/after the user's zero-slippage mint, though this is a secondary MEV extraction method.	Highly active pools.

Proof of Concept:

1. **Attacker** observes a pending `mint` transaction with `amount0Min = 0`.
2. **Front-run:** Attacker swaps `X` of `token0` -> `token1`, shifting the price significantly.
3. **Victim Tx:** `mint` executes at the manipulated price, using `0` for the minimum checks. The user deposits `amount0Desired` and `amount1Desired`, receiving a skewed position.
4. **Back-run:** Attacker swaps back `token1` -> `token0` at the restored price.
5. **Result:** Attacker did not profit from the spread but causing depositor to effectively overpays for their position.

Mitigation:

Consider enforce positive values (> 0) or, at minimum, require that at least one of `amount0Min` or `amount1Min` is positive, with a clear opt-out via 1 `wei` for advanced users.

```
1     function mint(MintParams calldata params)
2         ...
3     {
4 +     require(amount0Min > 0 && amount1Min > 0, 'Min amounts must be
> 0');
5     ...
6
7     function increaseLiquidity(IncreaseLiquidityParams calldata params)
8         ...
9     {
10 +    require(amount0Min > 0 && amount1Min > 0, 'Min amounts must be
> 0');
11        Position storage position = _positions[params.tokenId];
12    ...
13
14    function decreaseLiquidity(DecreaseLiquidityParams calldata params)
15        ...
16    {
17 +    require(amount0Min > 0 && amount1Min > 0, 'Min amounts must be
> 0');
18        require(params.liquidity > 0);
```

Informational

[I-1]: Non-Atomic Deployment and Initialization of Pools Creates Front-Running Risk

Descriptions:

`UniswapV3PoolDeployer::deploy` Uniswap V3 uses `CREATE2` to deterministically deploy pools via `UniswapV3Factory::createPool`. The deployed pool is not immediately usable as it requires a separate call to `UniswapV3Pool.initialize(sqrtPriceX96)` to set the initial price and unlock the contract.

Critically, `initialize` is a public, permissionless function. This is intentional—it preserves Uniswap’s decentralized ethos, allowing anyone to bootstrap any pool without centralized approval.

Proof of Concept:

This creates a front-running window for users who deploy a pool in Transaction A and initialize it in Transaction B. An attacker can:

1. A bot is listening to mempool for Transaction A,
2. The bot submits a front-running transaction before Transaction B, calling `initialize` with a malicious initial price (e.g., 100x the fair price).
3. The user’s subsequent `initialize` call reverts because the pool is already initialized.
4. The user is forced to either accept the manipulated pool or deploy a new one at additional cost.

Mitigation:

Deploy and initialize the pool atomically, UniswapV3 documentation should note and emphasis the importance of this through `PoolInitializer::createAndInitializePoolIfNecessary`.

[I-2]: PoolAddress::computeAddress uses precomputed POOL_INIT_CODE_HASH will not work on chains that has non-standard CREATE2 implementations, leading to invalid address computed

Descriptions:

SwapRouter.sol uses computeAddress that rely on the standard EIP-1014 CREATE2 formula to verify if msg.sender is the UniswapV3 pool address during several callback functions. Some chains like ZKSync uses zkEVM bytecodes (zksolc) which differs from standard EVM bytecode, which will result in different POOL_INIT_CODE_HASH and using it to compute the pool address will not result in the correct address. This issue affects not only ZKsync Era but any chain where the Uniswap V3 core contracts are recompiled with a different compiler version, optimizer settings, or custom EVM implementation. The POOL_INIT_CODE_HASH must be recalculated for each unique deployment artifact.

```
1 PoolAddress.sol:
2 library PoolAddress {
3     bytes32 internal constant POOL_INIT_CODE_HASH = 0
4         xe34f199b19b2b4f47f68442619d555527d244f78a3297ea89325f843f87b8b54
5         ;
6     ...
7     function computeAddress(address factory, PoolKey memory key)
8         internal pure returns (address pool) {
9         require(key.token0 < key.token1);
10        pool = address(
11            uint256(
12                keccak256(
13                    abi.encodePacked(
14                        hex'ff',
15                        factory,
16                        keccak256(abi.encode(key.token0, key.token1,
17                            key.fee)),
18                        POOL_INIT_CODE_HASH
19                    )
20                )
21            )
22        );
23    }
24 }
```

List of places computeAddress is used:

- SwapRouter.sol#L48
- NonfungiblePositionManger.sol#L272
- NonfungiblePositionManger.sol#L324
- NonfungibleTokenPositionDescriptor.sol#L59

- `CallbackValidation.sol#L33`
- `PositionValue.sol#L119`
- `PairFlash.sol#L127`
- `LiquidityManagement.sol#L63`

Impact: If the deployed chains has non-standard `CREATE2` implementations, i.e. ZKSync Era, it may break most `v3-periphery` interactions (write) with `v3-core`:

1. `addLiquidity` to compute the wrong pool address, potentially sending tokens to the wrong contract or reverting.
2. `uniswapV3MintCallback` verification to fail, as the computed address will not equal `msg.sender`, causing swaps to revert.

Proof of Concept:

Refer to ZKSync official system-contracts documentation [here](#).

Mitigation:

If it is being deployed on Ethereum, Arbitrum, Optimism, or Base, no remediation is required. The code functions as intended and is battle-tested. Else, consider the following:

1. Implementing `UniswapV3Factory.sol::getPool` for pool address verification.
2. Verify the deployed bytecode hash on each chain. For ZKSync, The protocol must use the adapted `era-uniswap-v3-periphery` contracts or modify `computeAddress` to use the chain's specific address derivation formula. The `POOL_INIT_CODE_HASH` must also be verified against the deployed bytecode on that chain.